المدرسة الوطنية للعلوم التطبيقية
ⵜⵉⵎⵙⵙⵓⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⵏⴰⵡⵉⵏ ⵜⵉⵜⵜⴰⵡⵉⵏ
Ecole Nationale des Sciences Appliquées de Fès

جامعة سيدي محمد بن عبد الله
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵙⵉⴷⵉ ⵎⵓⵃⴰⵎⵎⴷ ⴱⵏ ⵄⴱⴷⵍⵍⴰⵀ
Université Sidi Mohamed Ben Abdellah

TP1 :

Automatisation Réseau via APIs RESTful

BELHAJ KHAOULA

N 9

GSCSI- G1

Table des matières

Table des matières 2
Table des figures 3
Introduction 4
Mise en place de l'environnement 5
1. Installer Postman 5
2. Désactiver la vérification SSL 5
3. Compte Cisco DevNet 6
4. Créer une Collection Postman 7
Synthèse des Exercices 8
EX 1 : Premiers pas avec une API publique 8
Question 1.1 — Récupérer la liste des utilisateurs 8
Question 1.2 — Récupérer un seul utilisateur 9
Question 1.3 — Tester une ressource inexistante 9
EX 2 : Connexion à un vrai switch Cisco via RESTCONF 10
Question 2.1 — Tester la connexion 10
Question 2.2 — Que se passe-t-il sans authentification ? 10
Question 2.3 — Découvrir les capacités du switch 11
EX 3 : Lire les interfaces d'un switch Cisco via RESTCONF 13
Question 3.1 — Lister toutes les interfaces 13
Question 3.2 — Lire une interface spécifique 14
Question 3.3 — Cibler uniquement l'IPv4 15
EX 4 : Modifier la configuration via RESTCONF 16
Question 4.1 — Créer une interface Loopback (POST) 16
Question 4.2 — Modifier la description avec PATCH 17
Question 4.3 — Supprimer l'interface (DELETE) 17
Tableau de bord des méthodes HTTP utilisées 19
Conclusion 20

Table des figures

Figure 1 Installer Postman 5
Figure 2 Désactiver la vérification SSL 6
Figure 3 Catalyst 9000 Always-On 6
Figure 4 l'onglet I/O 7
Figure 5 Basic Auth 7
Figure 6 Headers 7
Figure 7 Le code de statut 200 OK 8
Figure 8 User 3 9
Figure 9 User 999 9
Figure 10 Tester la connexion 10
Figure 11 No Auth 11
Figure 12 capabilities 11
Figure 13 Lister toutes les interfaces 13
Figure 14 Lire une interface spécifique 14
Figure 15 Cibler uniquement l'IPv4 15
Figure 16 Loopback 16
Figure 17 204 No Content 17
Figure 18 404 Not Found 18

Introduction

L'évolution des infrastructures réseaux vers le **Software-Defined Networking (SDN)** a transformé la manière dont les ingénieurs interagissent avec les équipements. Historiquement dépendante de l'interface en ligne de commande (CLI), la gestion du réseau s'oriente désormais vers l'utilisation d'interfaces de programmation applicative (**APIs**).

Ce TP a pour objectif de maîtriser l'utilisation des **APIs RESTful** pour administrer un commutateur Cisco Catalyst 9000 de manière programmable. À travers une progression pédagogique, nous avons d'abord exploré les concepts fondamentaux du protocole HTTP avec des APIs publiques, pour ensuite nous attaquer à la configuration réelle d'un équipement via **RESTCONF**. Cette approche permet de comprendre comment les formats de données structurés (JSON) et les modèles de données (YANG) facilitent l'automatisation, la surveillance et la modification de configurations à grande échelle, remplaçant ainsi les scripts de "screen-scraping" instables par des échanges de données robustes et prédictibles.

Mise en place de l'environnement

Cette étape a marqué l'entrée dans l'environnement Cisco DevNet.

- **Manipulation technique :** Désactivation de la vérification SSL (pour accepter les certificats auto-signés du lab) et mise en place de la **Basic Auth**.
- **Sécurité :** Nous avons observé que sans authentification, le switch retourne un code **401 Unauthorized**. C'est une barrière critique pour protéger l'accès au plan de gestion (Management Plane) de l'équipement.
- **Headers :** L'ajout de `Accept: application/yang-data+json` est crucial pour indiquer au switch que nous souhaitons interpréter les données selon le modèle YANG en format JSON.

1. Installer Postman

Postman est une application graphique Windows qui permet d'envoyer des requêtes HTTP/HTTPS



Figure 1 Installer Postman

2. Désactiver la vérification SSL

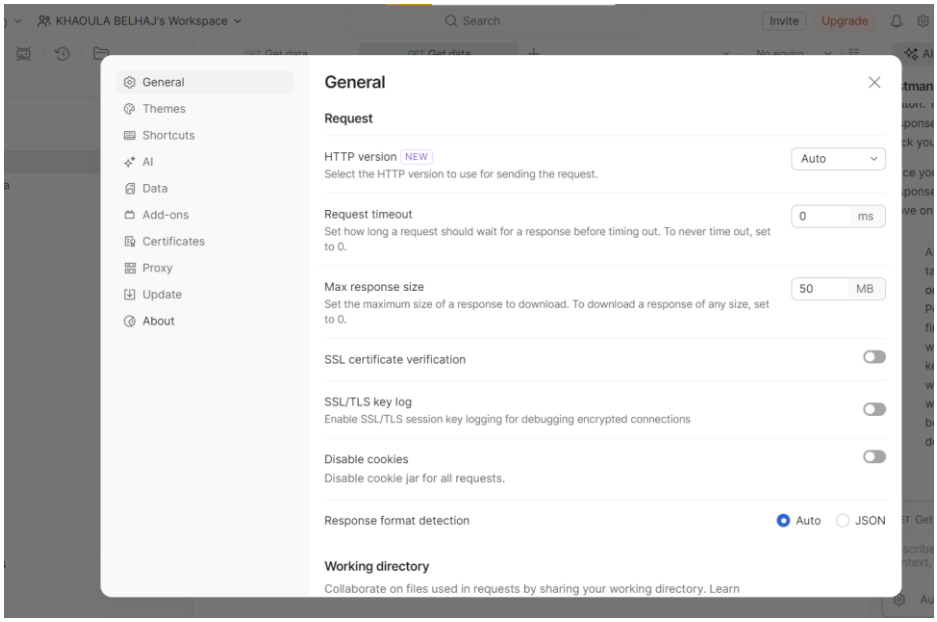


Figure 2 Désactiver la vérification SSL

3. Compte Cisco DevNet

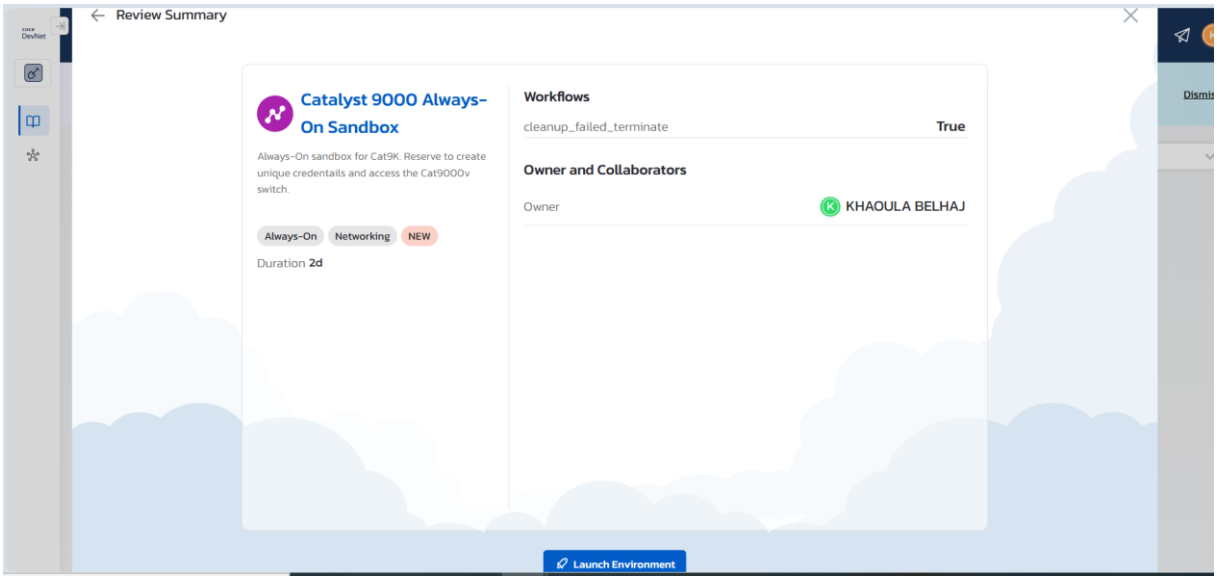


Figure 3 Catalyst 9000 Always-On

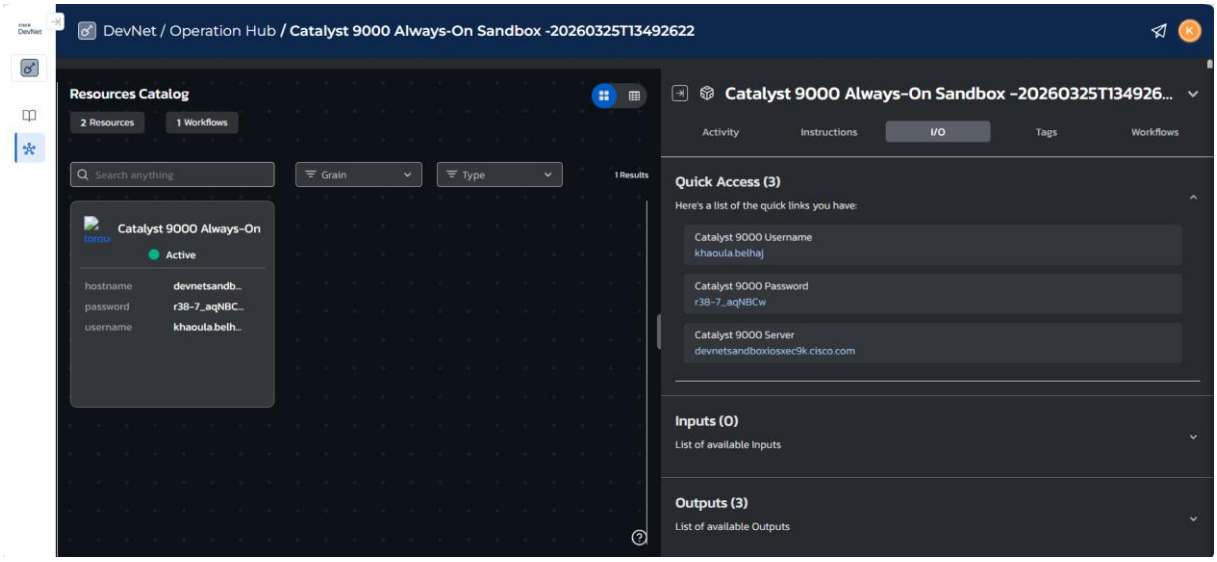


Figure 4 l'onglet I/O

4. Créer une Collection Postman

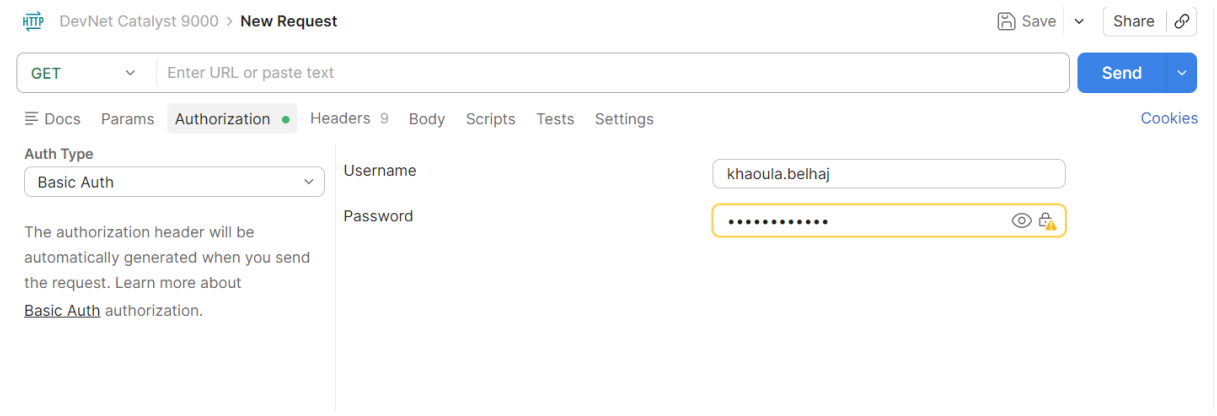


Figure 5 Basic Auth

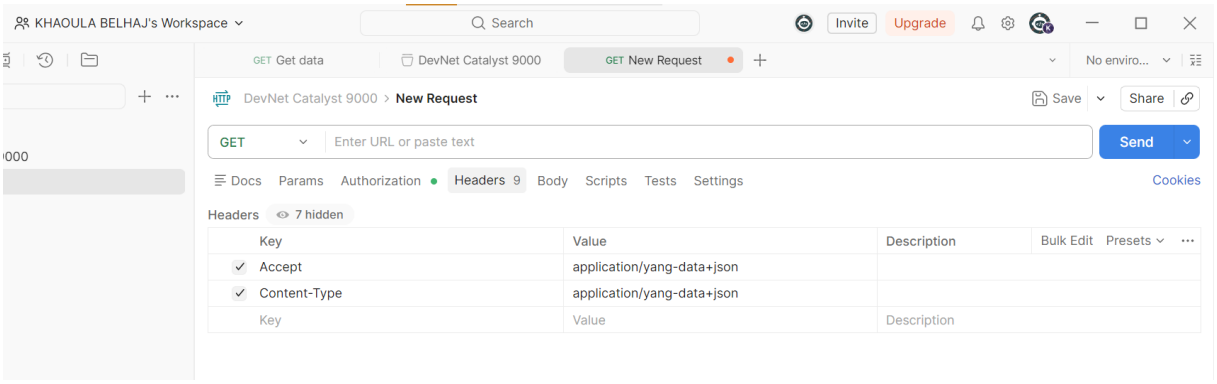


Figure 6 Headers

Synthèse des Exercices

EX 1 : Premiers pas avec une API publique

L'utilisation de *JSONPlaceholder* a permis de valider les bases avant d'accéder au matériel critique.

Question 1.1 — Récupérer la liste des utilisateurs

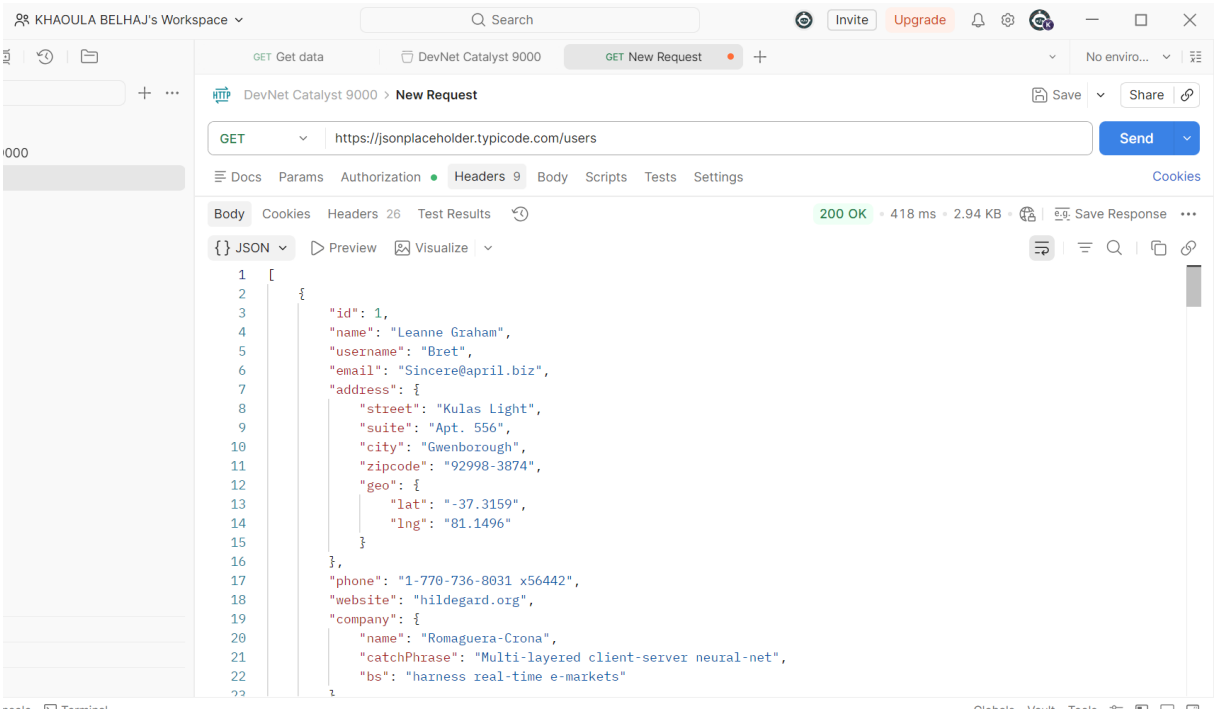


Figure 7 Le code de statut 200 OK

- **Concepts clés :** Utilisation de la méthode **GET**.
- **Résultats :** Le code de statut **200 OK** confirme la réussite.
- a) Code de statut HTTP

  200 OK

  b) Nombre d'utilisateurs retournés

  10 utilisateurs

  c) Format des données reçues

  JSON

Question 1.2 — Récupérer un seul utilisateur

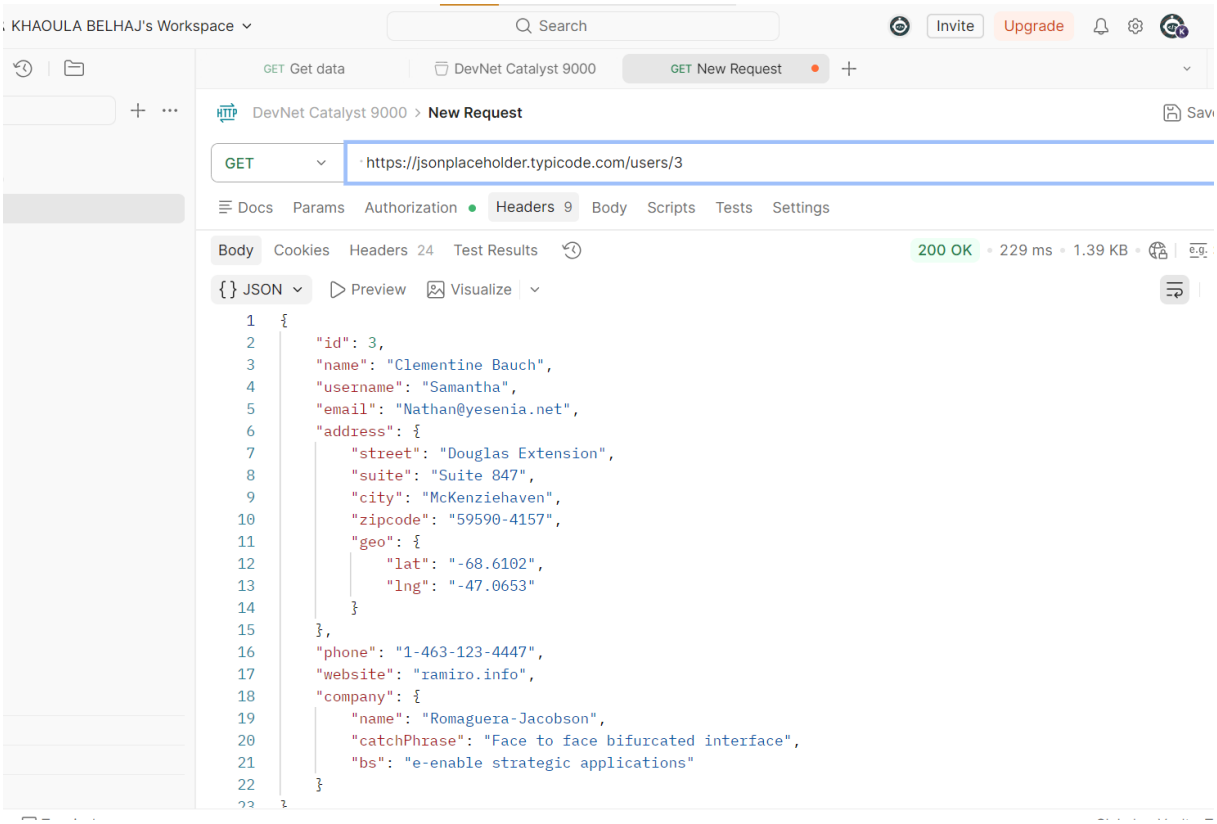


Figure 8 User 3

a) https://jsonplaceholder.typicode.com/users/3

b) "name": "Clementine Bauch", "username": "Samantha", "email": "Nathan@yesenia.net",

Question 1.3 — Tester une ressource inexistante

Compréhension du code **404 Not Found** lors de l'accès à une ressource inexistante (`/users/999`).

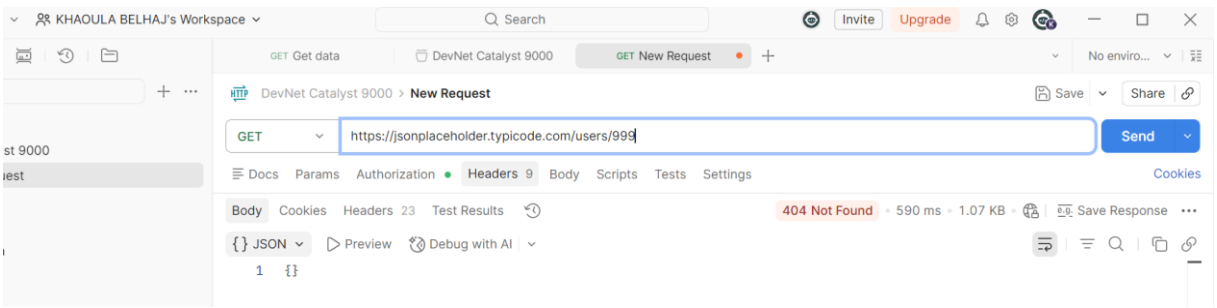


Figure 9 User 999

404 : La ressource demandée n’existe pas

EX 2 : Connexion à un vrai switch Cisco via RESTCONF

Question 2.1 — Tester la connexion

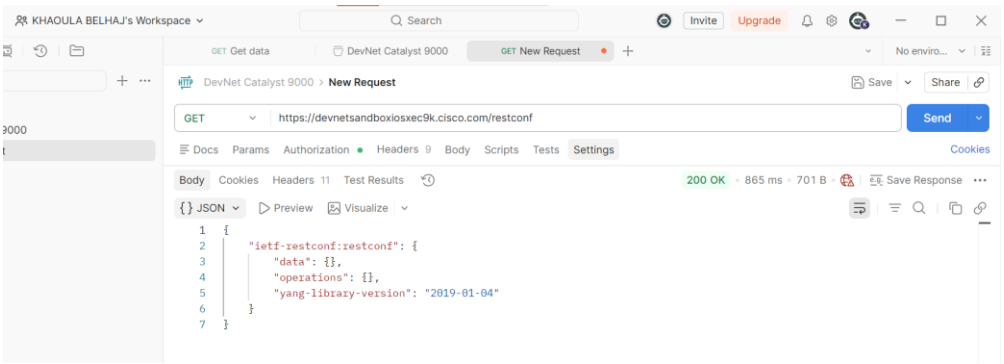


Figure 10 Tester la connexion

Envoie une requête GET vers :

https://devnetsandboxiosxec9k.cisco.com/restconf

a) Code de statut obtenu

200 OK

La connexion au switch fonctionne correctement avec authentification + headers

b) Si on supprime le header Accept

406 Not Acceptable

Question 2.2 — Que se passe-t-il sans authentification ?

Retire l'authentification (Authorization → Type : No Auth) et renvoie la même requête.

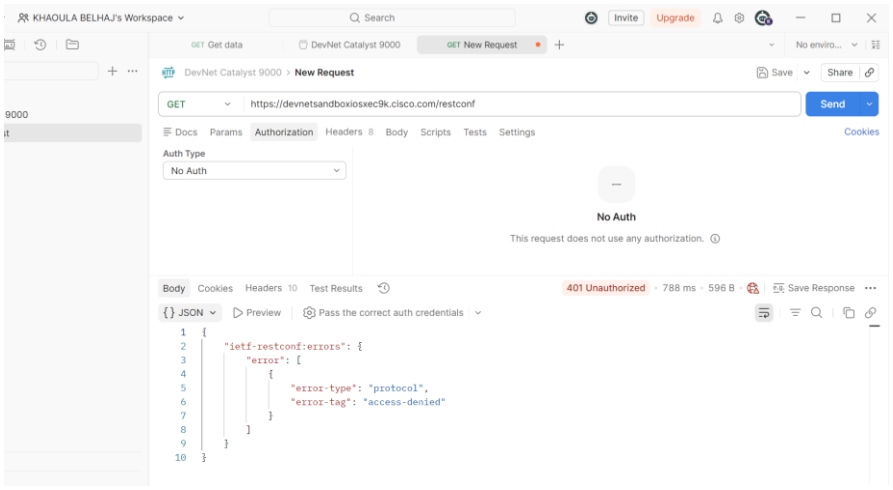


Figure 11 No Auth

a) Code 401 = Non autorisé

b) Il est important en production pour des raisons de **sécurité** :

• Empêche l'accès non autorisé aux équipements réseau

• Protège la configuration du switch (VLAN, interfaces, routing…)

• Évite les attaques (modification ou sabotage du réseau)

Question 2.3 — Découvrir les capacités du switch

Remets l'authentification et envoie une requête GET vers :

https://devnetsandboxiosxec9k.cisco.com/restconf/data/netconf-state/capabilities

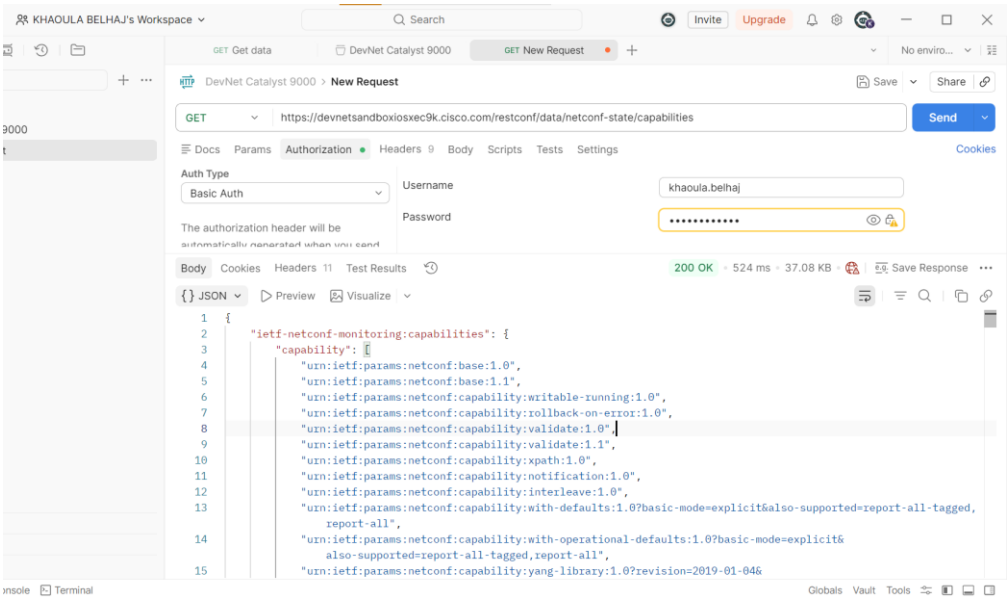


Figure 12 capabilities

a) Contenu de la réponse

Une liste de capacités NETCONF supportées par le switch sous format JSON

• modules YANG supportés

• versions des modèles

• fonctionnalités disponibles

b) À quoi servent ces capacités ?

Elles permettent de savoir :

• Ce que le switch peut faire

• Quels modèles YANG sont disponibles

• Quelles opérations sont supportées

EX 3 : Lire les interfaces d'un switch Cisco via RESTCONF

L'interrogation des interfaces a permis de comparer la vue API et la vue CLI.

- **Observation :** Une requête vers `/ietf-interfaces:interfaces` retourne une liste exhaustive des interfaces physiques et logiques.
- **Encodage :** Utilisation du code `%2F` pour remplacer le caractère `/` dans l'URL (ex: `GigabitEthernet0/0` devient `GigabitEthernet0%2F0`).
- **Analyse :** Contrairement à un `show ip interface brief` qui renvoie du texte non structuré, l'API renvoie un objet JSON hiérarchisé. C'est ce format qui permet à un script (Python, par exemple) d'extraire instantanément l'adresse IP sans avoir à "parser" du texte complexe.

Question 3.1 — Lister toutes les interfaces

Envoie une requête GET vers :

https://devnetsandboxiosxec9k.cisco.com/restconf/data/ietf-interfaces:interfaces

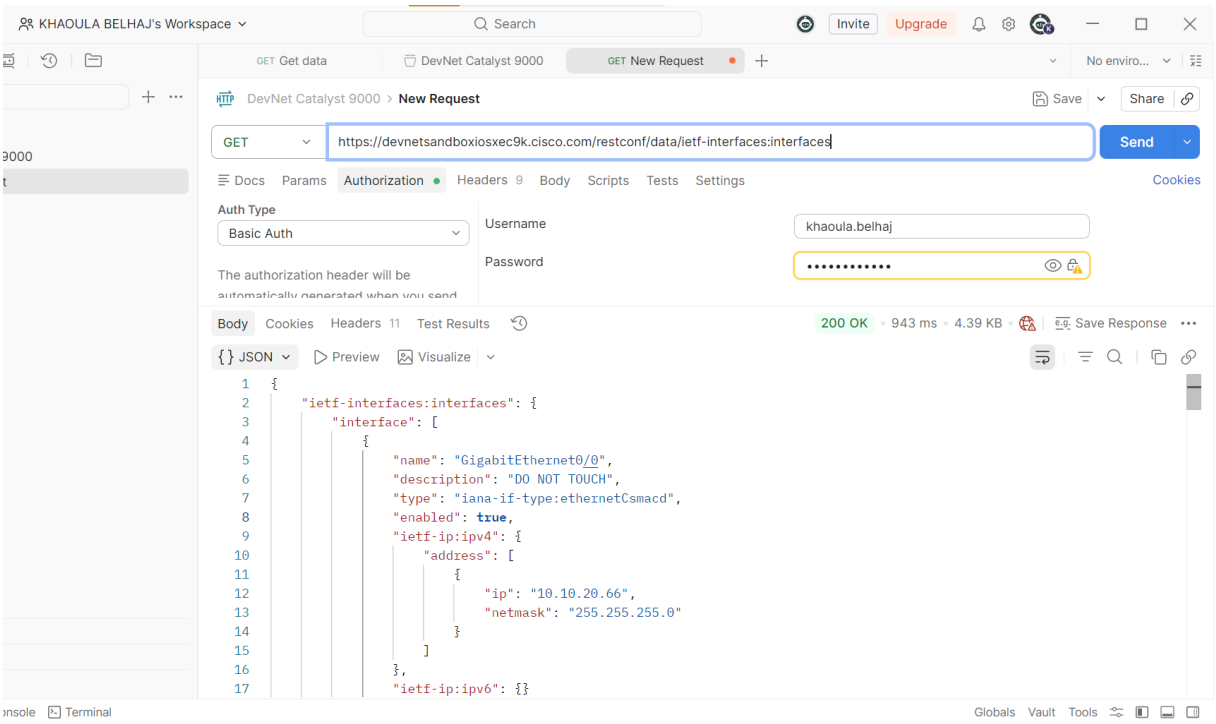


Figure 13 Lister toutes les interfaces

a) Combien d'interfaces le switch possède-t-il ?

Le nombre des interfaces est de 14 (GigabitEthernet + Loopback + Vlan…)

b) Quels types d'interfaces vois-tu (GigabitEthernet, Vlan...) ?

• **GigabitEthernet** → interfaces physiques

• **Vlan** → interfaces virtuelles (SVI)

• **Loopback** → interfaces logiques

c) Quelle est la structure JSON de la réponse ? Décris les clés principales.

- **interfaces** → conteneur principal
- **interface** → liste d'interfaces
- **name** → nom
- **type** → type d'interface
- **enabled** → état (up/down administrativement)
- **description** → description

Question 3.2 — Lire une interface spécifique

Récupère uniquement la configuration de l'interface de management :

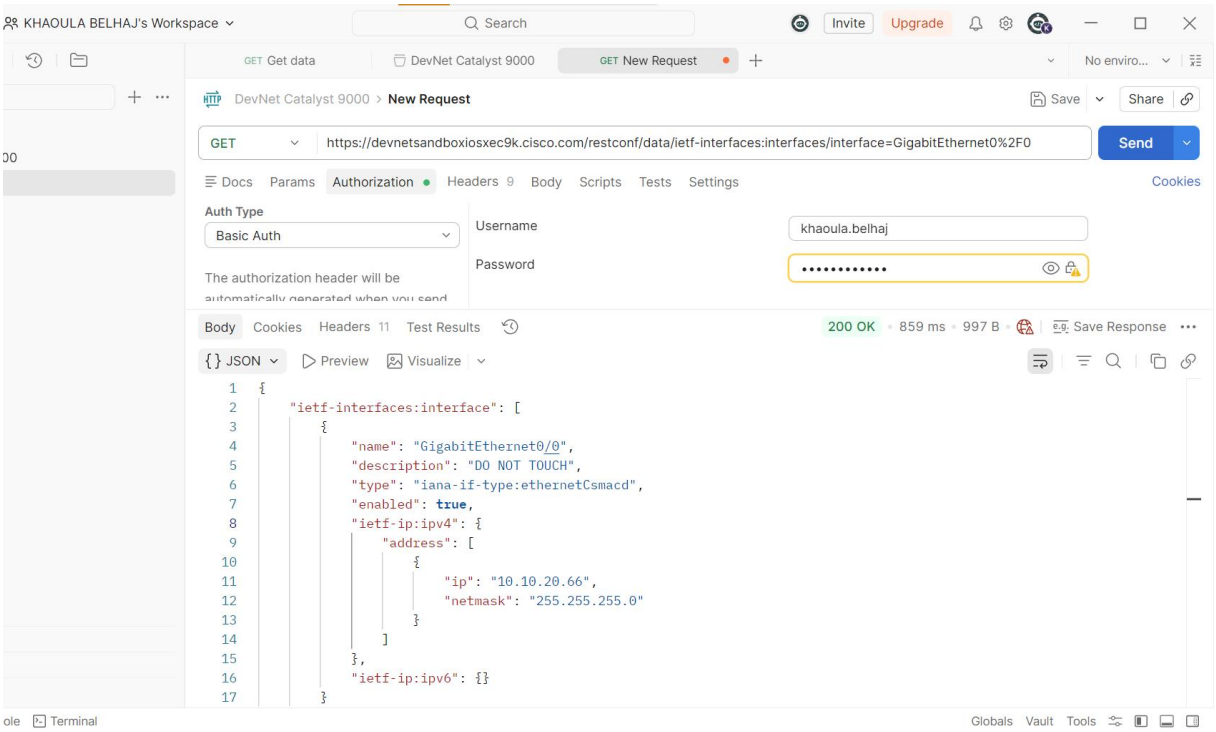


Figure 14 Lire une interface spécifique

Question 3.3 — Cibler uniquement l'IPv4

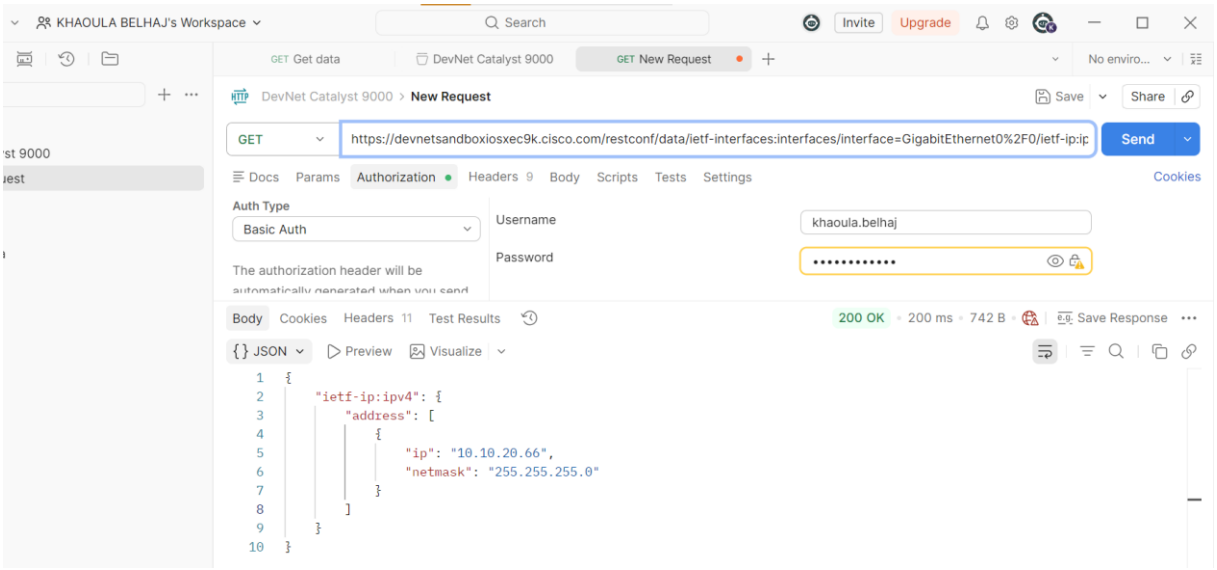


Figure 15 Cibler uniquement l'IPv4

Cette requête retourne **uniquement la partie IPv4,** Contrairement a la requête précédente qui retourne aussi la partie IPv6

EX 4 : Modifier la configuration via RESTCONF

C'est la phase active d'administration.

- **POST :** Création d'une nouvelle interface `Loopback100`. Le code de statut reçu est **201 Created**.
- **PATCH vs PUT :** Le **PATCH** a été utilisé pour modifier uniquement la description.
    - *Note technique :* Contrairement au PUT qui remplace toute la ressource, le PATCH ne modifie que les champs envoyés dans le corps de la requête, préservant ainsi l'adresse IP déjà configurée.
- **DELETE :** Suppression de la ressource. Un second envoi de la requête DELETE retourne une erreur (404 ou 409) car la ressource n'existe plus, démontrant l'aspect "stateless" et logique de l'API.

Question 4.1 — Créer une interface Loopback (POST)

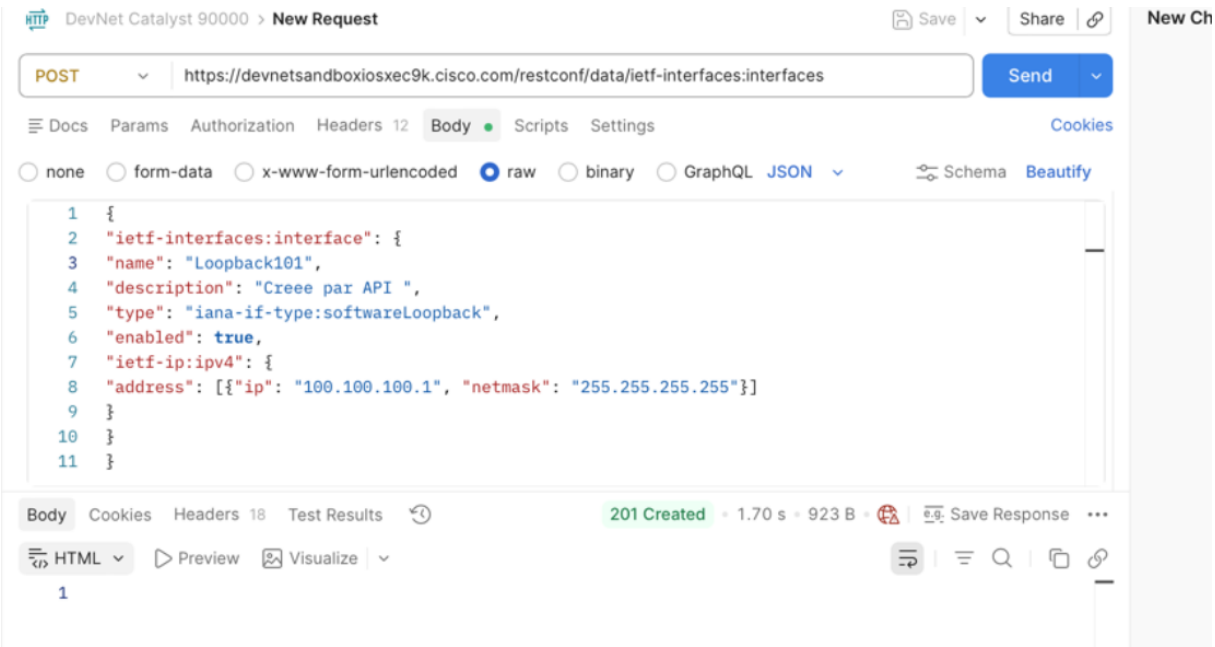


Figure 16 Loopback

a) Code de statut obtenu

201 Created

b) Vérification avec GET

Requête :

GET /interfaces/interface=Loopback101

• "name": "Loopback101"

• "description": "Creee par API"

• "enabled": true

• "ietf-ip:ipv4" avec IP 100.100.100.1

Question 4.2 — Modifier la description avec PATCH

a) Différence entre PUT et PATCH

PUT

• Remplace toute la ressource

PATCH

• Modifie uniquement la description

b) L'adresse IP a-t-elle changé ?

Non, elle ne change pas

• PATCH ne modifie que :

"description": "Description modifiee par PATCH"

Question 4.3 — Supprimer l'interface (DELETE)

a) Code de statut obtenu

204 No Content : suppression réussie

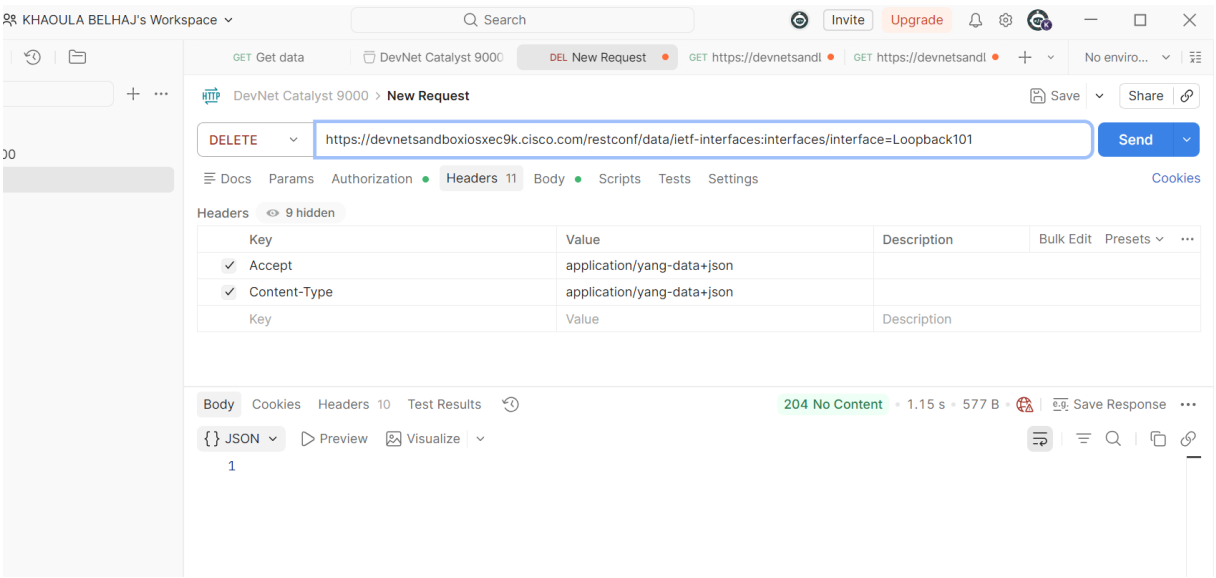


Figure 17 204 No Content

b) Si on refais DELETE une deuxième fois on obtient

404 Not Found

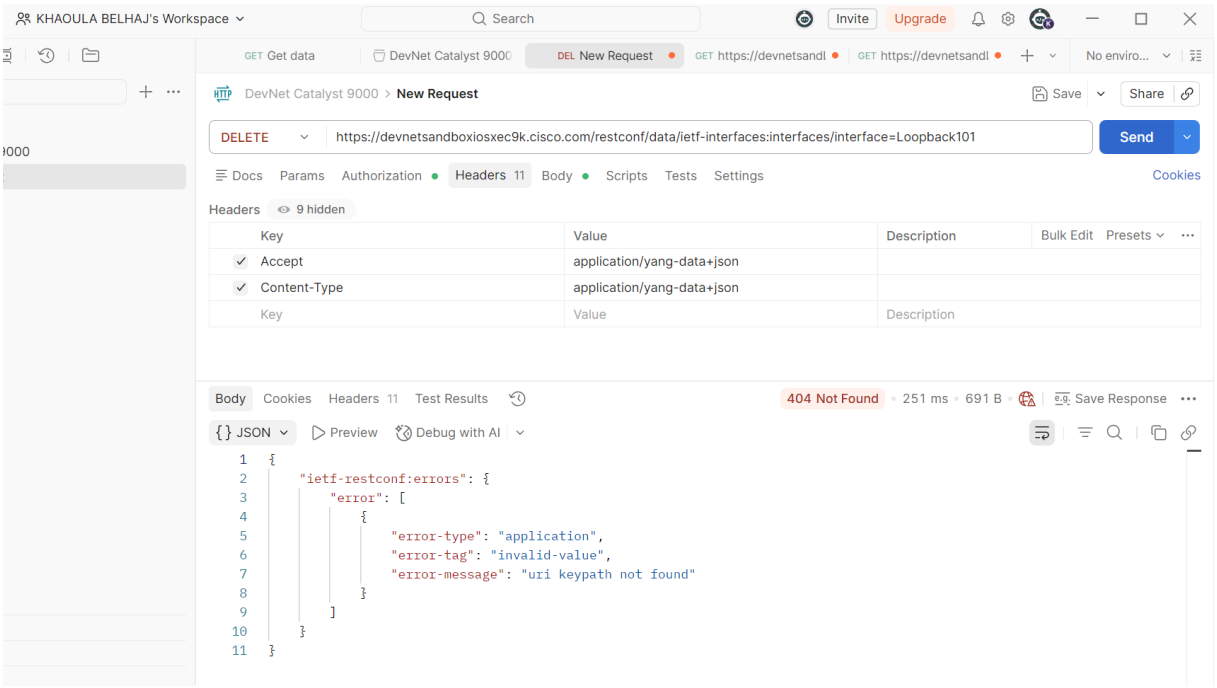


Figure 18 404 Not Found

Vu que :

• L'interface n'existe plus

• Donc impossible de la supprimer à nouveau

Tableau de bord des méthodes HTTP utilisées

Méthode	Action sur le Switch	Code Succès Type
GET	Lecture de la configuration ou de l'état	200 OK
POST	Création d'une nouvelle ressource (ex: Interface)	201 Created
PATCH	Modification partielle d'une configuration	204 No Content
DELETE	Suppression d'une interface ou d'une route	204 No Content

Conclusion

Ce cycle d'exercices démontre l'efficacité opérationnelle des APIs dans la gestion des réseaux modernes. En passant d'une gestion manuelle à une gestion via **RESTCONF**, nous avons pu constater trois avantages majeurs :

1. **La structure des données :** Le format JSON élimine les erreurs d'interprétation des données.
2. **La précision :** La capacité de cibler un sous-élément précis d'une interface (ex: uniquement l'IPv4) réduit la charge de traitement et la consommation de bande passante.
3. **Le potentiel d'automatisation :** Postman n'est qu'une étape. Les requêtes ainsi construites peuvent être directement exportées en code Python pour orchestrer des centaines d'équipements simultanément.

En conclusion, la maîtrise de RESTCONF et des modèles YANG est désormais une compétence indispensable pour tout administrateur réseau souhaitant évoluer vers les méthodologies **NetDevOps**. La prochaine étape logique consisterait à intégrer ces appels d'API dans des workflows d'automatisation via des outils comme Ansible ou des scripts Python utilisant la bibliothèque `requests`.